第三次上机实验题目

1、 子程序设计

设有 10 个学生的成绩分别是 76,69,84,90,73,88,99,63,100 和 80 分。试编制一个子程序统计 60~69 分，70~79 分，80~89 分，90~99 分和 100 分的人数并分别存放到 S6，S7，S8，S9 和 S10 单元中。

数据段中至少需要定义以下内容：

（1） ID db ‘2186123456’ (说明：以学号 2186123456 为例，此处应更换为自己的学号)

（2） array db ... (存放 10 个学生的成绩)

（3） 定义 S6，S7，S8，S9，S10 的内存单元

2、 高级汇编语言技术

试使用条件汇编和重复汇编编写一段程序，完成以下功能：根据给定名为 X 的字符串长度汇编以下指令，如果 X 的长度为 n，当 n<=5 时，汇编 n 次；当 n>5 时，汇编 6 次。

ADD AX，AX

数据段中至少需要定义以下内容：

（1） ID db ‘2186123456’ (说明：以学号 2186123456 为例，此处应更换为自己的学号)

（2） X 是一个目标字符串，根据下面的测试场景需求进行定义；

（3） 从这里定义自己认为必要的变量

测试场景要求：

（1） X 是一个 空字符串

（2） X 是一个长度为 5 的字符串，例如：'abcde' ；学号的后 5 位

（3） X 是一个长度为 10 的字符串，例如：'2186123456' ； 完整的学号

（4） 测试时，只能改变数据段中 X 的定义内容（通过注释符号），其它数据段、代码段中的内容必须保持不变。

实验要求：

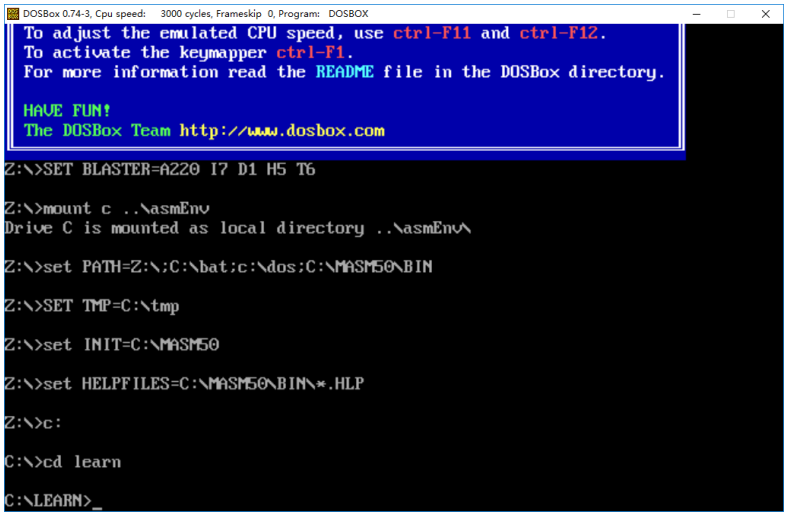
1、 在上机时间内，完成上述实验内容，并按要求在思源学堂**提交上机结果**，

具体格式和要求见 **提交上机结果的模板文件**；

2、 上机结果文件转换为 PDF 格式后再进行提交。

截图说明：

在 windows 下使用 Alt+Print 组合键可以对当前窗口进行截图，然后可以直接粘贴在这个文档中。这里以上机环境刚启动时为例进行说明，上机环境启动时的截图：



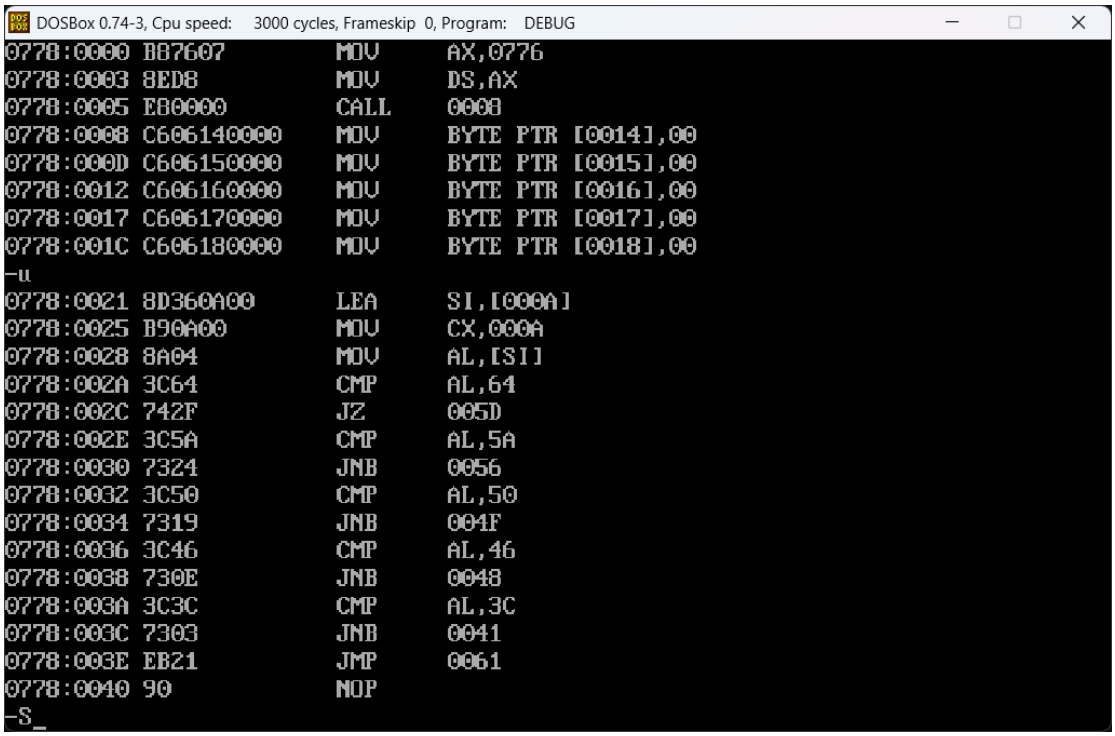
提交上机结果的模板文件

第 3 次上机

班级	学号	姓名
计算机 2301	2234412799	李祥熙

1、 子程序设计

（1）反汇编的截图



```
DOSBox 0.74-3, Cpu speed:   3000 cycles, Frameskip  0, Program:   DEBUG
0778:002E 3C5A          CMP     AL,5A
0778:0030 7324          JNB     0056
0778:0032 3C50          CMP     AL,50
0778:0034 7319          JNB     004F
0778:0036 3C46          CMP     AL,46
0778:0038 730E          JNB     0048
0778:003A 3C3C          CMP     AL,3C
0778:003C 7303          JNB     0041
0778:003E EB21          JMP     0061
0778:0040 90            NOP
-u
0778:0041 FE061400      INC     BYTE PTR [0014]
0778:0045 EB1A          JMP     0061
0778:0047 90            NOP
0778:0048 FE061500      INC     BYTE PTR [0015]
0778:004C EB13          JMP     0061
0778:004E 90            NOP
0778:004F FE061600      INC     BYTE PTR [0016]
0778:0053 EB0C          JMP     0061
0778:0055 90            NOP
0778:0056 FE061700      INC     BYTE PTR [0017]
0778:005A EB05          JMP     0061
0778:005C 90            NOP
0778:005D FE061800      INC     BYTE PTR [0018]
-S_
```

```
DOSBox 0.74-3, Cpu speed:   3000 cycles, Frameskip  0, Program:   DEBUG
0778:004F FE061600      INC     BYTE PTR [0016]
0778:0053 EB0C          JMP     0061
0778:0055 90            NOP
0778:0056 FE061700      INC     BYTE PTR [0017]
0778:005A EB05          JMP     0061
0778:005C 90            NOP
0778:005D FE061800      INC     BYTE PTR [0018]
-Su
  ^ Error
-u
0778:0061 46            INC     SI
0778:0062 E2C4          LOOP    0028
0778:0064 C3            RET
0778:0065 BB004C        MOV     AX,4C00
0778:0068 CD21          INT     21
0778:006A 50            PUSH    AX
0778:006B E84248        CALL    48B0
0778:006E 83C404        ADD     SP,+04
0778:0071 50            PUSH    AX
0778:0072 8DB67CFF      LEA     AX,[BP+FF7C]
0778:0076 50            PUSH    AX
0778:0077 E8020F        CALL    0F7C
0778:007A 83C404        ADD     SP,+04
0778:007D 8BB67CFF      MOV     SI,[BP+FF7C]
-S_
```

（2）在进行计算前，显示 ID、array 以及 S6——S10 的内存值的截图（多显示、少显示均扣分）

```
-d 0 18
0776:0000  32 32 33 34 34 31 32 37-39 39 4C 45 54 5A 49 58   2234412799LETZIX
0776:0010  63 3F 64 50 00 00 00 00-00                        c?dP.....
-S
```

（3）执行完计算后，显示 ID、array 以及 S6——S10 的内存值的截图（多显示、少显示均扣分）

```
-d 0 18
0776:0000  78 07 16 72 34 31 32 37-39 39 4C 45 54 5A 49 58   x..r412799LETZIX
0776:0010  63 3F 64 50 02 02 03 02-01                        c?dP.....
-Ss
```

（4）源代码

```
title Score Range Statistics


data segment
    ID      db  '2234412799'
    array   db  76,69,84,90,73,88,99,63,100,80

    S6      db  0               ; 60~69
    S7      db  0               ; 70~79
    S8      db  0               ; 80~89
    S9      db  0               ; 90~99
    S10     db  0               ; 100
data ends


code segment
    assume cs:code, ds:data
    main    proc
        ; assign the data segment base address to DS
        mov   ax, data
        mov   ds, ax

        ; TODO ...
        ; | add your code between arrows |
        ; v ---------------------------- v
```

```
    call  STAT_SCORE

; Count values in array into S6/S7/S8/S9/S10
STAT_SCORE  proc
    mov   S6, 0
    mov   S7, 0
    mov   S8, 0
    mov   S9, 0
    mov   S10, 0

    lea   si, array
    mov   cx, 10

NEXT_SCORE:
    mov   al, [si]

    cmp   al, 100
    je    IN_100

    cmp   al, 90
    jae   IN_90

    cmp   al, 80
    jae   IN_80

    cmp   al, 70
    jae   IN_70

    cmp   al, 60
    jae   IN_60
```

```
    jmp   SCORE_DONE

IN_60:
    inc   S6
    jmp   SCORE_DONE

IN_70:
    inc   S7
    jmp   SCORE_DONE

IN_80:
    inc   S8
    jmp   SCORE_DONE

IN_90:
    inc   S9
    jmp   SCORE_DONE

IN_100:
    inc   S10

SCORE_DONE:
    inc   si
    loop  NEXT_SCORE

    ret
STAT_SCORE  endp

    ; ^ ---------------------------- ^
```

```
        ; |          The END            |

        ; method 2: return to dos
        mov   ax, 4c00h
        int   21h
    main    endp
code ends
end main
```

2、 高级汇编语言技术

（1）场景 1 的.lst 文件的截图

ism M | condrept.asm ...\lab-3 M | condrept.asm ...\learn U | CONDREPT.LST U | empty.asm M

Assembly-Language > asmEnv > learn > CONDREPT.LST

```
Microsoft (R) Macro Assembler Version 5.00                  4/25/26 23:06:07
Conditional And Repeat Assembly Demo                        Page     1-1


                        ; Task 2: conditional assembly + repeat
                    assembly

                        title Conditional And Repeat Assembly D
                    emo

0000                    data segment
0000  32 32 33 34 34 31 32     ID      db  '2234412799'
      37 39 39

                        ; Test scenario: only change the de
                    finition content of X.
000A  00                X db 0                  ; scenario 1
                    : empty string
                        ; X db 'abcde',0        ; scenario 2
                    : length = 5
                        ; X db '2234412799',0   ; scenario 3
                    : length = 10

= 0000                  X_LEN   equ ($ - X - 1)
000B  0000              RESULT  dw  0
000D                    data ends

0000                    code segment
                        assume cs:code, ds:data

0000                    main    proc
0000  B8 ---- R                 mov   ax, data
0003  8E D8                     mov   ds, ax

0005  B8 0001                   mov   ax, 1

                        if X_LEN le 5
                            rept X_LEN
                                add ax, ax
                            endm
                        endif

0008  A3 000B R                 mov   RESULT, ax

000B  B8 4C00                   mov   ax, 4c00h
000E  CD 21                     int   21h
0010                    main    endp

0010                    code ends
                    end main
Microsoft (R) Macro Assembler Version 5.00                  4/25/26 23:06:07
Conditional And Repeat Assembly Demo                        Symbols-1


Segments and Groups:

                N a m e             Length   Align  Combine Class

CODE . . . . . . . . . . . . . .    0010    PARA    NONE
DATA . . . . . . . . . . . . . .    000D    PARA    NONE

Symbols:

                N a m e             Type     Value   Attr

ID . . . . . . . . . . . . . . .    L BYTE  0000    DATA

MAIN . . . . . . . . . . . . . .    N PROC  0000    CODE    Length = 0010

RESULT . . . . . . . . . . . . .    L WORD  000B    DATA

X  . . . . . . . . . . . . . . .    L BYTE  000A    DATA
X_LEN  . . . . . . . . . . . . .    NUMBER  0000

@FILENAME  . . . . . . . . . . .    TEXT  condrept


     43 Source  Lines
     43 Total   Lines
      9 Symbols

  50430 + 466018 Bytes symbol space free

      0 Warning Errors
      0 Severe  Errors
```

（2）场景 1 的反汇编的截图

```
-u0
0777:0000 B87607        MOV     AX,0776
0777:0003 8ED8          MOV     DS,AX
0777:0005 B80100        MOV     AX,0001
0777:0008 A30B00        MOV     [000B],AX
0777:000B B8004C        MOV     AX,4C00
0777:000E CD21          INT     21
0777:0010 FA            CLI
```

（3）场景 1 的显示 X 的内存值的截图（多显示、少显示均扣分）

```
-d a a
0776:0000                          00                          .
```

（4）场景 2 的.lst 文件的截图

ard.s | score.asm M | condrept.asm ...\lab-3 M | condrept.asm ...\learn U | CONDREPT.LST U

Assembly-Language > asmEnv > learn > CONDREPT.LST

```
Conditional And Repeat Assembly Demo                        Page      1-1

      1                 ; Task 2: conditional assembly + repeat
                      assembly
      2
      3                 title Conditional And Repeat Assembly D
                      emo
      4
      5 0000                data segment
      6 0000  32 32 33 34 34 31 32     ID      db  '2234412799'
      7       37 39 39
      8
      9                     ; Test scenario: only change the de
                      finition content of X.
     10                     ;X db 0                 ; scenario
                      1: empty string
     11 000A  61 62 63 64 65 00        X db 'abcde',0       ; scenario 2:
                      length = 5
     12                     ; X db '2234412799',0  ; scenario 3
                      : length = 10
     13
     14 = 0005                 X_LEN   equ ($ - X - 1)
     15 0010  0000             RESULT  dw  0
     16 0012                data ends
     17
     18 0000                code segment
     19                     assume cs:code, ds:data
     20
     21 0000                    main    proc
     22 0000  B8 ---- R                 mov   ax, data
     23 0003  8E D8                     mov   ds, ax
     24
     25 0005  B8 0001                   mov   ax, 1
     26
     27                         if X_LEN le 5
     28                             rept X_LEN
     29                                 add ax, ax
     30                             endm
     31 0008  03 C0                  1                 add ax, ax
     32 000A  03 C0                  1                 add ax, ax
     33 000C  03 C0                  1                 add ax, ax
     34 000E  03 C0                  1                 add ax, ax
     35 0010  03 C0                  1                 add ax, ax
     36                         endif
     37
     38 0012  A3 0010 R                 mov   RESULT, ax
     39
     40 0015  B8 4C00                   mov   ax, 4c00h
     41 0018  CD 21                     int   21h
     42 001A                    main    endp
     43
     44 001A                code ends
     45                 end main
Microsoft (R) Macro Assembler Version 5.00                  4/25/26 23:11:51
Conditional And Repeat Assembly Demo                        Symbols-1


Segments and Groups:

                N a m e             Length   Align  Combine Class

CODE . . . . . . . . . . . . . .    001A    PARA    NONE
DATA . . . . . . . . . . . . . .    0012    PARA    NONE

Symbols:

                N a m e             Type     Value   Attr

ID . . . . . . . . . . . . . . .    L BYTE  0000    DATA

MAIN . . . . . . . . . . . . . .    N PROC  0000    CODE    Length = 001A

RESULT . . . . . . . . . . . . .    L WORD  0010    DATA

X  . . . . . . . . . . . . . . .    L BYTE  000A    DATA
X_LEN  . . . . . . . . . . . . .    NUMBER  0005

@FILENAME  . . . . . . . . . . .    TEXT  condrept


     43 Source  Lines
     48 Total   Lines
      9 Symbols

  50430 + 466018 Bytes symbol space free

      0 Warning Errors
      0 Severe  Errors
```

Ln 1, Col 1 Spaces: 4 UTF-8 CRLF { } Plain Text Prettier

（5）场景 2 的反汇编的截图

```
C:\LEARN>debug condrept.exe
-u
0778:0000 B87607        MOV     AX,0776
0778:0003 8ED8          MOV     DS,AX
0778:0005 B80100        MOV     AX,0001
0778:0008 03C0          ADD     AX,AX
0778:000A 03C0          ADD     AX,AX
0778:000C 03C0          ADD     AX,AX
0778:000E 03C0          ADD     AX,AX
0778:0010 03C0          ADD     AX,AX
0778:0012 A31000        MOV     [0010],AX
0778:0015 B8004C        MOV     AX,4C00
0778:0018 CD21          INT     21
```

（6）场景 2 的显示 X 的内存值的截图（多显示、少显示均扣分）

```
-d a a
0776:0000                       61                      a
```

（7）源代码

```
title Conditional And Repeat Assembly Demo

data segment
    ID      db  '2234412799'

    ;X db 0                 ; scenario 1: empty string
    X db 'abcde',0       ; scenario 2: length = 5
    ; X db '2234412799',0  ; scenario 3: length = 10

    X_LEN   equ ($ - X - 1)
    RESULT  dw  0
data ends

code segment
    assume cs:code, ds:data

    main    proc
        mov   ax, data
        mov   ds, ax

        mov   ax, 1
```

```
        if X_LEN le 5
            rept X_LEN
                add ax, ax
            endm
        else
            rept 6
                add ax, ax
            endm
        endif

        mov   RESULT, ax

        mov   ax, 4c00h
        int   21h
    main    endp

code ends
end main
```